Contents

Chapter 19: Multi-Agent Systems

The previous chapters established a progression:

```
Chapter 15
Coding agent
    ↓
Chapter 16
Specification
    ↓
Chapter 17
Context engineering
    ↓
Chapter 18
Development loop
    ↓
Chapter 19
Multiple agents
```

Once a single agent can reason, use tools, manage context, execute code, and iterate through verification, the next obvious question is:

> **Should we use more than one agent?**

Sometimes the answer is yes.

Often it is no.

This distinction is extremely important.

Multi-agent systems are attractive because they appear to provide:

- parallelism
- specialization
- redundancy
- independent perspectives
- hierarchical decomposition
- improved fault tolerance
- separation of concerns

But they also introduce:

- coordination overhead
- communication cost
- duplicated work
- synchronization problems
- inconsistent state
- conflicting conclusions
- additional failure modes
- higher latency
- higher token consumption

Therefore:

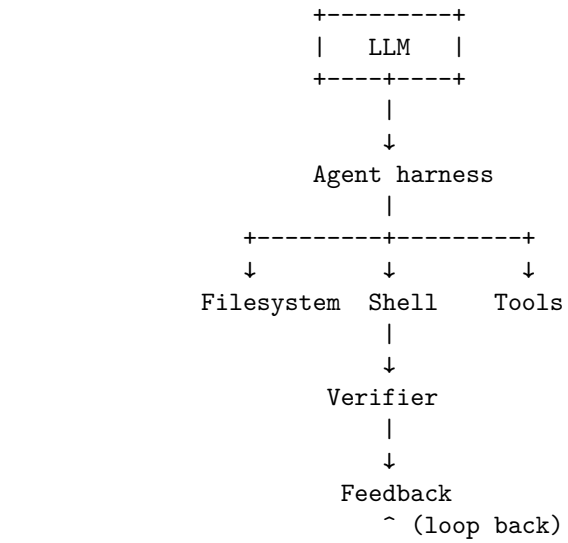
> **The goal of multi-agent engineering is not to maximize the number of agents. It is to determine whether multiple agents produce a better system than one well-designed agent.**

Agent multiplication is often cargo cult.

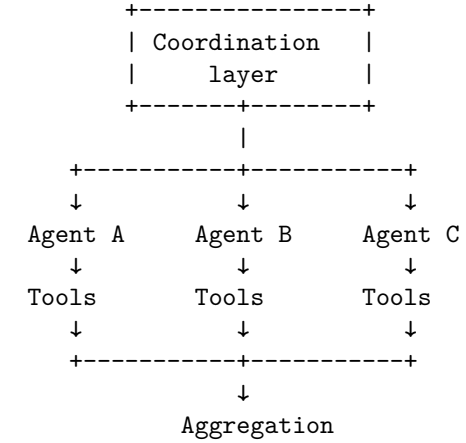
---

1. What Is a Multi-Agent System?

A simple coding agent can be modeled as:



A multi-agent system introduces multiple reasoning processes:



```
        ↓
  Verification
```

The agents may be:

- identical models with different contexts
- different models
- specialized agents
- different prompts over the same model
- agents operating on different tasks
- agents operating at different abstraction levels

The key architectural question is:

> **Why should these reasoning processes be separate?**

If there is no strong answer, a multi-agent architecture may simply add complexity.

---

2. One Agent vs. Multiple Agents

Suppose a task is:

> Add pagination to an existing REST endpoint.

A single agent can probably do this:

```
Task
 ↓
Agent
 ↓
Inspect code
 ↓
Modify implementation
 ↓
Run tests
 ↓
Fix
```

Introducing four agents might produce:

```
Manager
 |-- Architecture agent
 |-- Implementation agent
 |-- Test agent
 +-- Review agent
```

That sounds sophisticated.

But the system now has to coordinate:

```
Who owns the task?
Who has the latest source?
Who communicates changes?
Who resolves conflicts?
Who decides when the work is complete?
```

If the coordination cost exceeds the benefit of specialization, the multi-agent system is worse.

A useful conceptual comparison is:

$$Q_{\text{multi}} = Q_{\text{single}} + G_{\text{parallel}} + G_{\text{specialization}} + G_{\text{redundancy}} - C_{\text{coordination}} - C_{\text{communication}} - C_{\text{duplication}}$$

Use multiple agents only when the gains outweigh the costs.

---

3. The Four Fundamental Patterns

There are four particularly important multi-agent patterns:

1. **Parallel**
2. **Pipeline**
3. **Manager/worker**
4. **Critic**

These patterns represent different ways of distributing work and information.

---

4. Pattern 1 — Parallel Agents

The simplest pattern is parallel execution.

```
             +→ Agent A
             |
Task --------+→ Agent B
             |
             +→ Agent C
```

Each agent independently works on some aspect of the problem.

For example:

> Design an architecture for a document-processing system.

Run:

```
Agent A → architecture proposal
Agent B → security analysis
Agent C → scalability analysis
Agent D → cost analysis
```

Then aggregate the results.

```
Agent A -+
Agent B -+
Agent C -+→ Synthesizer → Final design
Agent D -+
```

This pattern works particularly well when subtasks are:

- independent
- decomposable
- computationally expensive
- naturally parallel
- independently verifiable

---

5. Parallelism and the Critical Path

Parallelism can reduce wall-clock time.

Suppose four independent tasks take:

$$T_1, T_2, T_3, T_4$$

Sequential execution costs approximately:

$$T_{\text{seq}} = T_1 + T_2 + T_3 + T_4$$

Parallel execution costs approximately:

$$T_{\text{parallel}} = \max(T_1, T_2, T_3, T_4) + T_{\text{coordination}}$$

The potential speedup is:

$$S = \frac{T_{\text{seq}}}{T_{\text{parallel}}}$$

But only if the tasks are genuinely independent.

If every agent constantly needs information from every other agent, parallelism disappears.

The architecture becomes a communication system rather than a parallel computation.

---

6. Parallel Agents for Independent Perspectives

Parallelism is not only about speed.

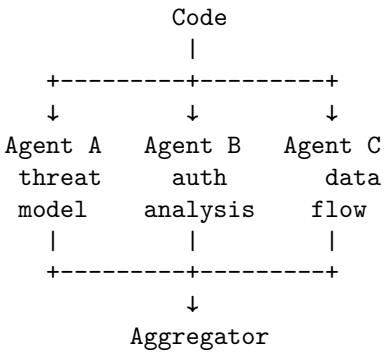
It can provide **diversity of reasoning**.

Suppose an agent must review code for security vulnerabilities.

Instead of one analysis:

```
Code
 ↓
Security agent
```

we can run:



If the agents make partially independent errors, multiple analyses can improve coverage.

This is analogous to ensemble methods in machine learning.

The benefit comes from **error diversity**.

If all agents make exactly the same mistake, adding agents provides little value.

---

7. Pattern 2 — Pipeline

In a pipeline, agents operate sequentially.

```
A → B → C → D
```

Each agent consumes the output of the previous agent.

For example:

```
Requirements
    ↓
Architect
    ↓
Developer
    ↓
Tester
    ↓
Reviewer
```

This resembles a software-development pipeline.

A concrete implementation might be:

```
Agent A:
Translate specification into architecture.

Agent B:
Translate architecture into implementation plan.

Agent C:
Implement the plan.

Agent D:
Review implementation.

Agent E:
Generate additional tests.
```

The advantage is specialization.

Each agent has a narrower responsibility.

---

8. Pipeline Failure Propagation

Pipelines have an important weakness:

> **Errors can propagate downstream.**

Suppose:

```
A → B → C → D
```

Agent A makes an architectural mistake.

Agent B accepts it.

Agent C implements it.

Agent D reviews implementation against the incorrect architecture.

The entire pipeline may produce a polished but fundamentally incorrect result.

This is a form of **correlated failure**.

Therefore pipelines need feedback paths:

```
A → B → C → D
^           |
+-----------+
```

or verification checkpoints:

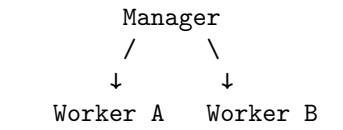
```
A → verify → B → verify → C → verify
```

Without them, specialization can amplify upstream errors.

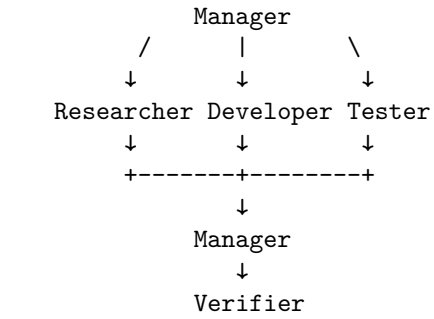
---

9. Pattern 3 — Manager/Worker

A manager agent decomposes the task and delegates work.



A larger system might look like:



The manager may be responsible for:

- decomposition
- task assignment
- prioritization
- state tracking
- integration
- conflict resolution
- stopping decisions

The workers focus on execution.

---

10. Manager/Worker Resembles Distributed Computing

The manager/worker pattern has a strong analogy to distributed systems.

```
Manager
   ↓
Task queue
   ↓
Workers
```

```
   ↓
Results
   ↓
Aggregator
```

Now familiar distributed-systems problems appear:

- task scheduling
- retries
- idempotency
- timeouts
- partial failure
- duplicate execution
- result ordering
- state consistency
- resource allocation

This is an important lesson:

> **Multi-agent systems are not merely prompt architectures. They are distributed systems whose workers happen to be reasoning models.**

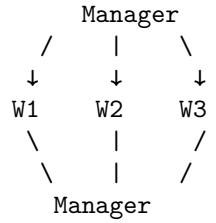
The same engineering disciplines apply.

---

11. The Manager Can Become a Bottleneck

The manager itself can become a single point of failure.

Consider:



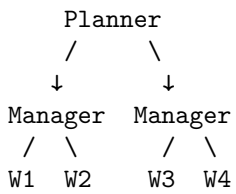
Every decision passes through it.

This can create:

- latency
- context overload
- token costs
- centralized decision errors

The manager may also become responsible for too much state.

A better design might distribute some coordination:



But now the architecture becomes more complicated.

Again:

> **Every additional agent introduces another architectural decision.**

---

12. Pattern 4 — Critic

A particularly useful pattern is:

```
Developer → Critic → Developer
```

The developer produces an artifact.

The critic evaluates it.

The developer receives the feedback and revises.

For example:

```
Specification
      ↓
Developer
      ↓
Implementation
      ↓
Critic
      ↓
Review findings
      ↓
Developer
      ↓
Revised implementation
```

This resembles the development loop from Chapter 18.

The difference is that the critic is an explicit reasoning component.

---

13. Critic Agents and Independent Evaluation

A critic can evaluate:

- correctness
- architecture
- security
- performance
- style
- requirements coverage
- edge cases

For example:

```
Developer:
"Implemented authorization."

Critic:
"Authorization occurs after document retrieval.
This creates a cross-tenant data exposure risk."

Developer:
"Move authorization into the repository query."

Critic:
"Now authorization is enforced before retrieval."
```

This can be powerful.

But the critic should ideally have some independence from the developer.

Otherwise:

```
Developer model
       ↓
Critic with same assumptions
       ↓
"Looks good."
```

may simply reproduce the same blind spot.

---

14. Adversarial Criticism

A stronger critic can be explicitly instructed to search for failure.

Instead of:

> Review the implementation.

use:

> Assume this implementation contains a serious defect. Find the strongest counterexample.

This changes the objective from:

```
confirm correctness
```

to:

```
attempt falsification
```

This is closely related to scientific reasoning.

The developer proposes.

The critic attempts to disprove.

The verifier provides evidence.

The developer revises.

---

15. Multi-Agent Systems and Diversity

The strongest justification for multiple agents is often **diversity**.

Suppose:

$$E_A$$

is the error made by Agent A and

$$E_B$$

is the error made by Agent B.

If:

$$P(E_A \cap E_B)$$

is substantially smaller than:

$$P(E_A)$$

then independent agents can provide useful redundancy.

But if:

$$E_A \approx E_B$$

then adding Agent B provides little additional information.

This gives a critical design question:

> **Are the agents actually independent in their failure modes?**

Changing the agent's name from "developer" to "reviewer" does not automatically create independence.

---

16. Independence Can Come From Different Contexts

Agents can be diversified through different information.

For example:

```
Developer:
implementation + specification

Security reviewer:
implementation + threat model

Performance reviewer:
implementation + benchmark requirements

Test designer:
specification + existing tests
```

Now each agent has a different perspective.

This is more useful than simply running four copies of the same prompt.

Context diversity can produce reasoning diversity.

---

17. Independence Can Come From Different Objectives

Agents can also have different optimization objectives.

```
Developer:
maximize functionality

Security agent:
minimize security risk

Performance agent:
minimize latency

Cost agent:
minimize infrastructure cost

Reviewer:
maximize requirements coverage
```

These objectives naturally create productive disagreement.

The system then needs an integration mechanism.

---

18. The Integration Problem

Multiple agents produce multiple outputs.

Now somebody must answer:

What is the final answer?

Suppose:

```
Agent A:
Use PostgreSQL.

Agent B:
Use DynamoDB.

Agent C:
Use Redis + PostgreSQL.
```

How does the system decide?

Possible mechanisms include:

Voting

```
A → PostgreSQL
B → DynamoDB
C → PostgreSQL

Winner: PostgreSQL
```

Manager synthesis

```
Manager
 ↓
Compare arguments
 ↓
Select architecture
```

Evidence-based selection

```
Proposal
 ↓
Benchmark
 ↓
Security test
 ↓
Cost analysis
 ↓
Winner
```

The third approach is often stronger because it moves the decision from opinion toward evidence.

---

19. Consensus Is Not Correctness

A dangerous assumption is:

> "If three agents agree, the answer must be correct."

Not necessarily.

If all three share the same incorrect assumption:

```
A → wrong
B → wrong
C → wrong
```

then:

```
3 votes != truth
```

Consensus is useful evidence only when the agents have sufficiently independent reasoning or when their conclusions are grounded in external verification.

This is why multi-agent systems should still have verifiers.

---

20. Multi-Agent Systems Need a Shared World Model

Agents need some representation of shared state.

For example:

```
Task state:
- authentication implemented
- 3 tests failing
- database migration pending
- security review incomplete
```

Without shared state, agents can work from inconsistent assumptions.

One agent may believe:

```
authentication uses OAuth
```

while another believes:

```
authentication uses API keys
```

This is a distributed consistency problem.

Possible solutions include:

- shared task state
- event logs
- artifact stores
- databases
- version control
- message queues
- structured agent outputs

---

21. Structured Communication

Agent-to-agent communication should ideally be structured.

Instead of:

```
"I think you should probably update the auth stuff..."
```

use:

```
{
  "finding": "Authorization occurs after retrieval",
  "severity": "critical",
  "location": "src/retrieval/query.py",
  "recommendation": "Apply tenant filter before database retrieval",
  "evidence": ["test_cross_tenant_access"]
}
```

Structured communication provides:

- predictable interfaces
- easier parsing
- validation
- persistence
- observability
- lower ambiguity

The same principle applies to agent communication as to conventional APIs.

---

22. Agent Interfaces

A mature multi-agent system should define explicit contracts. For example:

```
Architect Agent
Input:
    Specification
    Constraints
Output:
    Architecture
```

```
    Interfaces
    ADRs
    Risks

Developer Agent
Input:
    Specification
    Architecture
    Relevant repository context
Output:
    Code changes
    Tests
    Implementation notes

Reviewer Agent
Input:
    Specification
    Architecture
    Diff
    Tests
Output:
    Findings
    Severity
    Required changes
```

Now the agents resemble services in a distributed system.

---

23. Shared State vs. Isolated State

There is an important architectural choice.

Shared state All agents see the same evolving repository.

```
        Agent A
          ↓
     Shared repository
       \       /
   Agent B   Agent C
```

Advantages:

- simple artifact sharing
- immediate visibility
- easy integration

Risks:

- race conditions

- accidental interference
- inconsistent intermediate states

Isolated state Each agent works on a branch or workspace.

```
          Repository
         /    |     \
        ↓     ↓      ↓
     Branch A B      C
       ↓     ↓      ↓
     Agent A B      C
```

Advantages:

- isolation
- independent experimentation
- easier rollback

Risks:

- merge conflicts
- integration complexity
- duplicated work

This is another distributed-systems tradeoff.

---

24. Multi-Agent Coding Systems and Git

Version control becomes particularly useful. Imagine:

```
main
 |-- agent/security
 |-- agent/performance
 +-- agent/implementation
```

Each agent can independently modify the code. The integration agent then evaluates:

```
implementation
+
security changes
+
performance changes
```

This creates a natural architecture for parallel software engineering. However, merging code is itself a nontrivial reasoning problem. The system must verify:

```
tests
security
performance
```

```
behavior
architecture
```

after integration.

---

25. When Multi-Agent Systems Actually Make Sense

Multiple agents are particularly useful when at least one of the following is true.

1. Tasks are naturally parallel

```
Research A
Research B
Research C
```

2. Tasks require different expertise

```
Developer
Security specialist
Performance specialist
```

3. Independent verification is valuable

```
Generator
    ↓
Independent critic
```

4. The problem is too large for one context

```
Large repository
 ↓
Subsystem agents
```

5. Work has different time scales

```
Fast worker:
code search
Slow worker:
benchmark
Background worker:
documentation analysis
```

6. Failure isolation matters One agent can experiment without contaminating another's workspace.

---

26. When Multi-Agent Systems Do Not Make Sense

Avoid multiple agents when:

The task is small

```
Rename a variable.
```

One agent is enough.

The subtasks are highly coupled If every agent constantly needs every other agent's output, parallelism creates communication overhead.

The task is sequential by nature Some reasoning must happen in a strict order.

There is no meaningful specialization Five identical agents may simply produce five similar answers.

Verification is already strong If a deterministic test suite can reliably determine correctness, adding several critics may add little value.

Coordination dominates computation If agents spend more effort communicating than solving the problem, the architecture is counterproductive.

---

27. Amdahl's Law for Agents

Parallel agent systems are constrained by the same principle as parallel computing. Suppose fraction P of the workload can be parallelized. With N agents, idealized speedup is bounded by:

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

This is Amdahl's Law. If only 50% of the task is parallelizable:

$$P = 0.5$$

then even infinitely many agents provide at most:

$$S(\infty) = 2$$

before coordination overhead. In real systems:

$$S_{\text{real}} < S_{\text{Amdahl}}$$

because of:

- communication
- synchronization
- scheduling
- duplicated work
- model latency
- token costs

Therefore:

> **More agents do not imply proportionally more capability or speed.**

---

28. Token Economics

Multi-agent systems can become expensive quickly. Suppose one agent consumes:

$$T$$

tokens. Five agents might consume:

$$5T$$

before accounting for:

- coordination
- synthesis
- repeated context
- retries
- manager reasoning

A more realistic cost model is:

$$C_{\text{total}} = \sum_i C_i + C_{\text{coordination}} + C_{\text{synthesis}} + C_{\text{retries}}$$

This matters because many multi-agent architectures accidentally solve a problem by spending 5–10x more inference budget. The correct question is therefore:

> **What additional capability do we get per additional unit of inference cost?**

---

29. Agent Multiplication as Cargo Cult

There is a recurring failure mode in AI engineering:

```
Single agent
    ↓
Add manager
```

```
    ↓
Add planner
    ↓
Add researcher
    ↓
Add reviewer
    ↓
Add critic
    ↓
Add evaluator
    ↓
"Now we have an advanced agent system."
```

But if the original problem was simply poor context or inadequate verification, none of these additions may solve it. The architecture becomes more complicated without becoming more capable. This is **agent multiplication without a demonstrated systems benefit**. The remedy is empirical evaluation.

---

30. The Right Experimental Question

Do not ask:

> "Would more agents make this system better?"

Ask:

> **"What measurable capability do additional agents provide that one agent cannot provide at acceptable cost?"**

For example:

```
Metric:
security vulnerabilities found
Single agent:
8/12 found
Single + security critic:
11/12 found
Single + 3 generic agents:
8/12 found
```

Now the security critic has a demonstrated value. But:

```
Single:
92% success
Three generic agents:
93% success
Cost:
4.2x higher
```

```
Latency:
3.1x higher
```

This may not justify the architecture.

---

31. The Agent Ablation Test

Ablation is one of the best tools for evaluating multi-agent architectures. Start with:

```
System:
One agent
```

Then add one component at a time:

```
+ critic
+ planner
+ parallel researcher
+ manager
```

Measure after every addition. For example:

Architecture	Success	Cost	Latency
Single agent	78%	1.0x	1.0x
+ planner	83%	1.3x	1.2x
+ critic	90%	1.8x	1.6x
+ researcher	91%	2.6x	2.2x
+ second researcher	91%	3.5x	3.0x

The results tell you where additional agents stop providing meaningful returns.

---

32. The Marginal Value of an Agent

We can define the marginal benefit of agent i as:

$$\Delta Q_i = Q_{\text{system}+i} - Q_{\text{system}}$$

and its marginal cost as:

$$\Delta C_i = C_{\text{system}+i} - C_{\text{system}}$$

A useful design criterion is:

$$\frac{\Delta Q_i}{\Delta C_i}$$

If this ratio becomes very small, additional agents are probably not justified. This gives a quantitative way to resist architectural fashion.

---

33. Multi-Agent Systems Should Still Have a Verifier

One of the most important architectural principles is:

```
Agents
  ↓
Proposal
  ↓
Independent verification
  ↓
Acceptance
```

not:

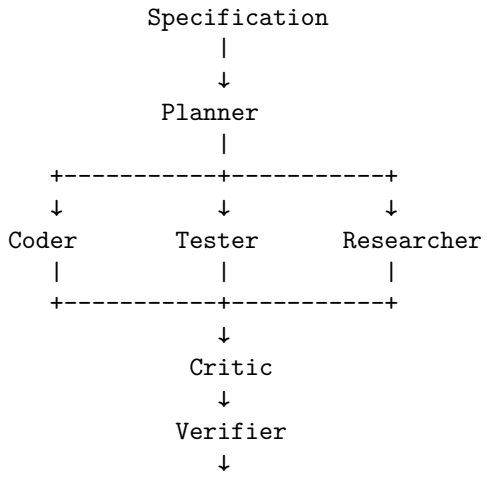
```
Agent A
 ↓
Agent B
 ↓
Agent C
 ↓
"Looks good."
```

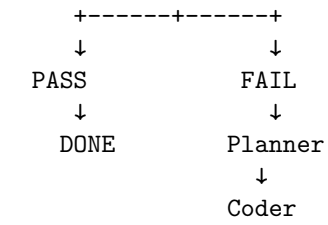
The first architecture grounds the system in evidence. The second risks creating an echo chamber. A multi-agent system does not eliminate the need for verification. It makes verification even more important.

---

34. A Practical Multi-Agent Coding Architecture

A reasonable first implementation is:



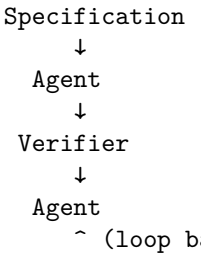


Notice that the agents are not the ultimate authority. The verifier is.

---

35. A More Minimal Architecture

However, start simpler. A surprisingly powerful architecture may be:



Only introduce another agent if the evaluation shows a real weakness. For example:

```
Single agent fails security review
                ↓
Add security critic
                ↓
Security failures decrease
```

Now the second agent has a demonstrated purpose.

---

36. Multi-Agent Systems as Organizational Design

There is a deeper analogy. Traditional engineering organizations divide work among:

```
architects
developers
test engineers
security engineers
SREs
product managers
```

Multi-agent systems reproduce some of this organizational structure computationally. But there is an important difference:

> **Software organizations have communication costs, and so do agent organizations.**

Every handoff creates potential information loss. Therefore the architecture should minimize unnecessary boundaries. This is analogous to Conway's Law:

> System architecture tends to reflect communication structures.

A multi-agent system with ten tightly coupled agents can easily become a distributed monolith.

---

37. Agent Boundaries Should Follow Information Boundaries

A strong principle for decomposition is:

> **Create an agent boundary where there is a meaningful boundary in information, responsibility, or expertise.**

Good boundary:

```
Application development
        |
        |-- Security analysis
        +-- Performance analysis
```

Poor boundary:

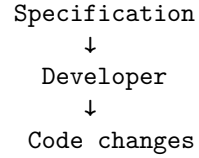
```
Frontend developer
Backend developer
API developer
Database developer
```

when every change requires coordination across all four. The latter may simply create distributed complexity.

---

38. Build One

For the Chapter 19 project, build a simple **Developer + Critic** system.



```
        ↓
     Critic
        ↓
Review findings
        ↓
    Developer
        ↓
   Verifier
        ↓
 PASS / FAIL
```

The Developer should:

- inspect the repository
- implement the feature
- run tests
- modify code

The Critic should:

- inspect the specification
- inspect the diff
- inspect relevant tests
- identify defects
- identify missing requirements
- identify security risks
- propose corrections

The Verifier should remain independent.

---

39. Instrument the System

Measure:

```
single-agent success rate
multi-agent success rate
iterations
token usage
cost
latency
defects discovered
defects introduced
regressions
critic findings
critic false positives
critic false negatives
```

Then run the same benchmark with:

```
A. Single agent
B. Developer + critic
C. Developer + two critics
D. Developer + generic second agent
```

This gives you an empirical answer to:

> **Does multi-agent architecture actually improve the system?**

---

40. The Deeper Principle

The progression from Chapter 15 through Chapter 19 can now be summarized:

```
Coding Agent
     ↓
Specification
     ↓
Context
     ↓
Feedback Loop
     ↓
Multiple Reasoning Processes
```

But every additional layer introduces complexity. The objective is therefore not:

$$\max(\text{agents})$$

It is:

$$\max \frac{\text{verified capability}}{\text{cost} + \text{latency} + \text{complexity} + \text{risk}}$$

This is the correct engineering objective.

---

41. Key Takeaways

1. **Multi-agent systems are not automatically better than single-agent systems.**
2. **Use multiple agents when there is a concrete reason:** parallelism, specialization, independent verification, context isolation, or failure containment.
3. **The four fundamental patterns are:**

   ```
   Parallel
   A -+
   B -+→ Result
   C -+
   ```

```
Pipeline
A → B → C → D

Manager/Worker
    Manager
    /     \
   W1     W2

Critic
Developer → Critic → Developer
```

4. **Parallelism is useful only when work is genuinely independent.** Otherwise coordination overhead can dominate.

5. **Pipelines provide specialization but can propagate upstream errors.** Verification checkpoints and feedback paths are essential.

6. **Manager/worker systems are distributed systems.** They introduce scheduling, consistency, retry, timeout, partial-failure, and coordination problems.

7. **Critic agents are useful when they provide genuinely independent evaluation.**

8. **Agent diversity matters more than agent count.** Multiple agents are valuable when their failure modes, contexts, expertise, or objectives differ.

9. **Consensus is not correctness.** Three agents agreeing can still produce the same wrong answer.

10. **Multi-agent systems require explicit communication contracts.** Structured inputs, outputs, shared state, and artifact interfaces reduce ambiguity.

11. **Multi-agent systems should still use independent verification.** Agents should propose; verifiers should establish evidence of correctness.

12. **Every agent has a cost.**

$$C_{\text{total}} = \sum_i C_i + C_{\text{coordination}} + C_{\text{synthesis}} + C_{\text{retries}}$$

13. **Use ablation studies to justify additional agents.** Add one component at a time and measure whether it improves meaningful system metrics.

14. **The right question is not "How many agents should we use?"** It is:

"What capability does this additional agent provide, and can we demonstrate that capability empirically?"

15. **Agent multiplication without measured benefit is cargo cult engineering.**

16. **A good starting architecture is often just:**

```
Agent
  ↓
Verifier
  ↓
Agent
  ^ (loop back)
```

Add a second agent only when the evaluation identifies a specific limitation that the additional agent can address.

17. **The fundamental optimization problem is:**

$$\max \frac{\text{verified capability}}{\text{cost} + \text{latency} + \text{complexity} + \text{risk}}$$

The deepest lesson of Chapter 19 is therefore not how to build multi-agent systems. It is how to **justify not building one**. A single well-contextualized, well-specified agent with a strong verification loop will often outperform a complicated collection of loosely coordinated agents. The engineering maturity comes from knowing the difference.